一、数据结构的基础

1.什么是数据? data

数据是描述客观事物属性的数，字符以及所有能输入到计算机中，并被计算机程序识别和处理的符号的集合

数据必要的两个前提：

1)能够被输入到计算机中

2)能够被计算机处理

2.数据元素 data element

是数据的基本单位，在计算机中通常作为一个整体考虑和处理的

eg:学生成绩表

```
学号   姓名   分数        <---管理数据
1     张三    80          <---数据元素
2     李四    90          <---数据元素
3     王五    85
    eg:
        数据元素：张三的所有信息

转化成C语言的描述
struct stu
{
    int id;
    char name[50];
    int score;
};

struct stu s[3];
```

3.数据项 data item

是描述数据的最小单位，一个数据元素是由多个数据项组成的，即数据元素的某个属性就是一个数据项

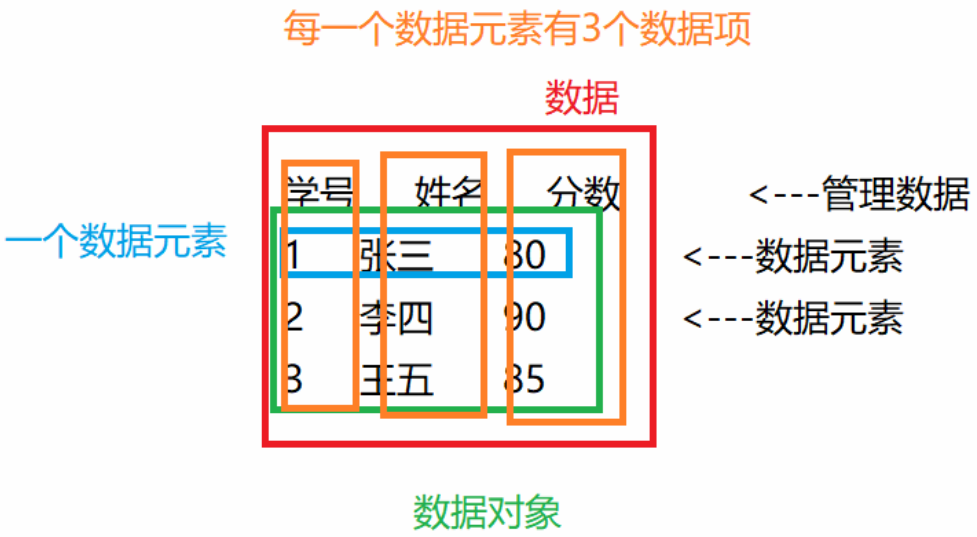
数据项是数据不可分割的最小单位

eg:

```
日期：2026年3月23日
      xxxx年x月xx日
日期由年 月 日组成
日期 是 数据元素
年 月 日 是 数据项
```

4.数据对象 data object

是性质相同的数据元素的集合，是数据的子集



关系：

数据 包含 ->数据对象 包含 -> 数据元素 -> 数据项

二、数据结构的分类

1)逻辑结构

```
指数据对象中，数据元素之间的相互关系(逻辑关系)
从逻辑关系上，描述数据，它对数据的存储是无关的，是独立于计算机存储的

eg:
    学号   姓名   分数
    1      张三    80
    2      李四    90
    3      王五    85

    按照分数从小到大排列，也可以按照分数从大到小排列，也可以按照学号排列
    跟你存储数据的顺序可能一样也可能不一样
```

```
1.集合
    结构中的数据元素，除了 同属于一个集合 外 别无其他关系
    eg:11 4374 238 4894

2.线性关系   数组(顺序表) 和   链表(链式线性表) 栈   队列        一对一
    数据元素之间的关系是 “线性”的   关系连成一条直线
    集合中每个元素(除了第一个元素)都有一个唯一的前驱(前一个)元素
    集合中每个元素(除了最后一个元素)都有一个唯一的后继(后一个)元素
    eg:
        int a[4] = {11,12,13,14};
        a[0] a[1] a[2] a[3]

3.非线性结构
```

```
    树形结构(层次结构)：文件系统、族谱        树与二叉树          一对多
    网状结构:地铁线路图                      图                  多对多
```

2)存储结构

数据结构在计算机中的表示称为数据的物理结构，也称为存储结构 -> 物理关系(地址关系)

存储结构(数据元素地址编号的关系)

```
0x3000   1
0x3004   2
0x3008   3

逻辑关系 按照从大到小排列
    0x3008   3
    0x3004   2
    0x3000   1
```

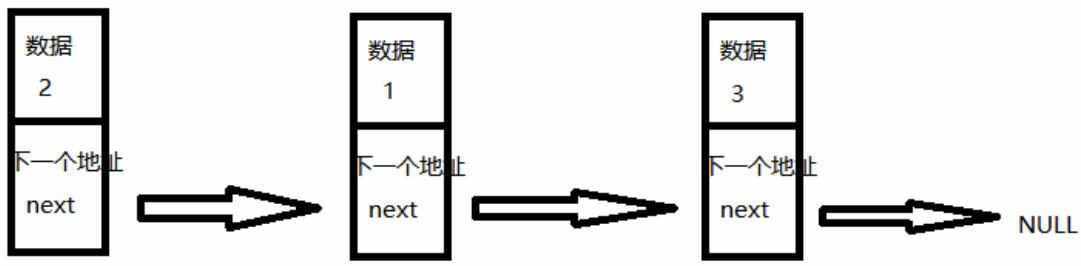
1.顺序存储（数组）

```
使用存储单元地址的物理关系描述他们的逻辑关系
逻辑关系在前面的元素，先存储
线性表的顺序存储表示是指用一组连续的存储单元依次存储线性表中的每一个元素
数据存储   逻辑关系和物理关系一致
eg:
    存储以下线性表(1,3,5,7)
    int a[100] = {1,3,5,7};
    int n = 4;//元素的个数的保存
    逻辑关系：1->3->5->7
    物理关系：1->3->5->7
    做了从大到小排序后
    int a[100] = {7,5,3,1};
    逻辑关系：7->5->3->1
    物理关系：7->5->3->1

基本操作：
    增 删 改 查

优缺点：
    优点：
        简单，查找方便，修改方便
    缺点：
        增加和删除一个元素不太方便(需要挪动大量元素)
        空间可能不足或浪费
```

2.链式结构(链式线性表 链表)

```
线性表的链式结构   指的是不一定要用连续的地址存储每一个元素(连续与否都可以)
在保存数据元素的同时，还需要存储它的下一个或上一个元素的地址

eg:
    存储
        0x3000   2
        0x3004   1
```

```
        0x3008   3
    从大到小
        head = 0x3008;//head指针存储第一个元素的地址
        0x3008   3   的下一个地址为 0x3000
        0x3000   2   的下一个地址为 0x3004
        0x3004   1   的下一个地址为 NULL
    转化为C语言的描述
    struct node
    {
        int data;   //存储数据
        struct node *next;  //存放下一个地址
    };

    //开辟空间
    struct node a = {2,NULL},b={1,NULL},c={3,NULL};
    //存储关系
    a.next = &b;
    b.next = &c;
    c.next = NULL;
    //排序
```



练习：用上面的链式结构，存储线性表(1,3,5,7),并且依次存储，存储完成后，用head指针保存第一个元素的地址，只通过head访问每一个元素的值，请打印链表中的数据

```
#include <stdio.h>

typedef struct node //数据结点
{
    int data;//数据域
    struct node *next;//指针域
}node;

int main()
{
    //创建4个结点
    node a,b,c,d;
    //为每个结点赋值以及关系的建立
    (&a)->data = 1;
    (&a)->next = &b;
    (&b)->data = 3;
    (&b)->next = &c;
    (&c)->data = 5;
    (&c)->next = &d;
    (&d)->data = 7;
    (&d)->next = NULL;
    //定义一个head指针保存第一个元素地址
    node *head = &a;   //head = &a
```

```
    /*
        //打印
        printf("%d\n",(&a)->data);//关系：head = &a    得到head->data
        printf("%d\n",(&b)->data);//关系：(&a)->next = &b; ====> (&a)->next->data
==> head->next->data
        printf("%d\n",(&c)->data);//关系：(&b)->next = &c; ====> (&b)->next->data
==> (&a)->next->next->data ==> head->next->next
        printf("%d\n",(&d)->data);

        node *temp = head;//移动指针
        printf("%d ",temp->data);//1
        temp = temp->next;//temp=head->next ===> temp = &b
        printf("%d ",temp->data);//3
        temp = temp->next;//temp=(&b)->next ===> temp = &c
        printf("%d ",temp->data);//5
        temp = temp->next;//temp=(&c)->next ===> temp = &d
        printf("%d ",temp->data);//7
    */
    node*temp = head;
    while(temp!=NULL)
    {
        //重复的代码
        printf("%d ",temp->data);
        temp = temp->next;//访问下一个元素
    }


}
```

3.索引存储结构

通常在存储结点信息的同时，还建立附加的索引表

索引表由若干索引项组成，每一个结点在索引表中都有一个索引项，

索引表由关键字和地址组成

eg:

```
索引表
    id      地址
    101     0x2000
    102     0x1000
    103     0x4000
    104     0x3000

数据区
    内存地址      id  姓名
    0x1000      102 李四
    0x2000      101 张三
    0x3000      104 赵六
    0x4000      103 王五
1.查索引表
    找到关键字104 ->对应的地址0x3000
2.去数据区获取数据
    到地址0x3000 ->获取赵六的数据即可
```

4.散列存储结构(哈希存储)

根据关键字，直接计算出该节点的存储地址

eg:

```
散列函数H(key) = 3*key / 2
key=2    H=2%5   -> 存储地址2
key=7    H=7%5   -> 存储地址2
key=3    H=3%5   -> 存储地址3

哈希表
地址0      地址1      地址2      地址3      地址4
5                     7          3
                      2
```

三、数据结构的实现

- 建立一个数据结构
- 消除一个结构
- 从一个数据机构中删除一个数据
- 把一个数据元素插入到一个数据结构中
- 对数据结构进行访问
- 对一个数据结构中的元素，进行修改
- 对一个数据结构进行查找

四、线性表

线性表是n个元素的有限序列集合(n>=0)，记作 L = (a0,a1,...an-1),其中a表示线性表中的数据元素(表项)，n是线性表的长度，L就是表名字

n=0，是一个空表，第一个表项称为：表头a0 最后一项成为：表尾an-1

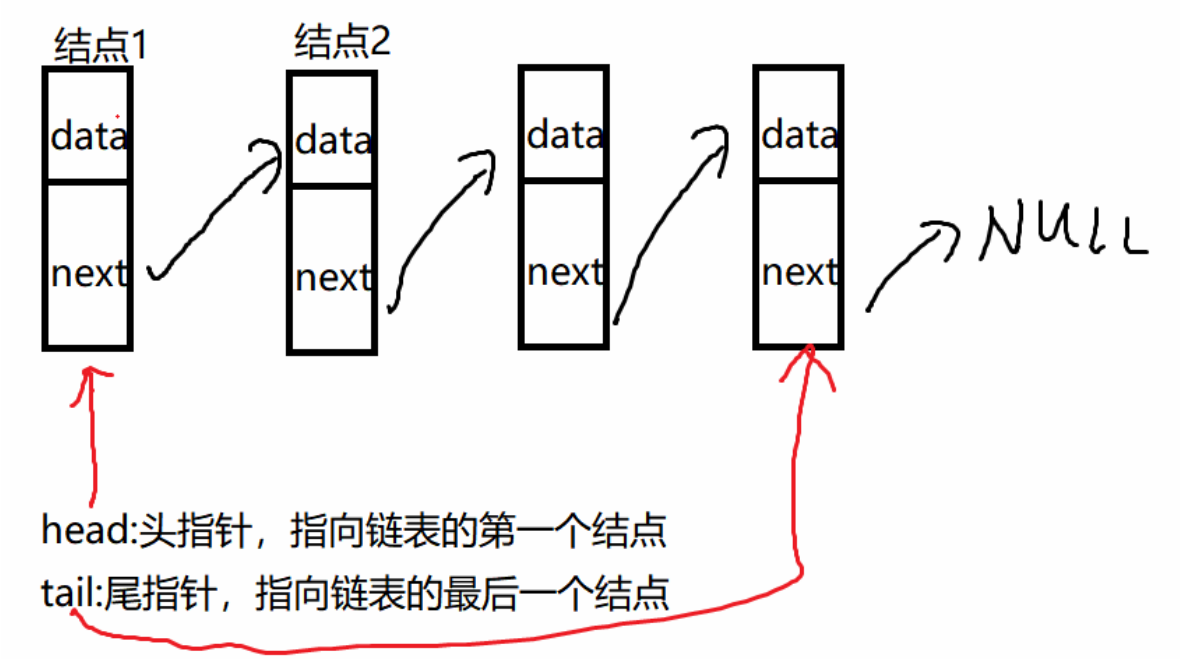
线性表的特点：

a.除了第一个元素外，其他的每个元素，有且仅有一个直接前驱

b.除了最后一个元素外，其他的每个元素，有且仅有一个直接后继

线性表主要分为:顺序表和链式表

五、单链表



用该结构体来描述结点

```
struct node
{
    int data;   //存储数据
    struct node *next;  //存放下一个地址
};
```

单链表的使用:

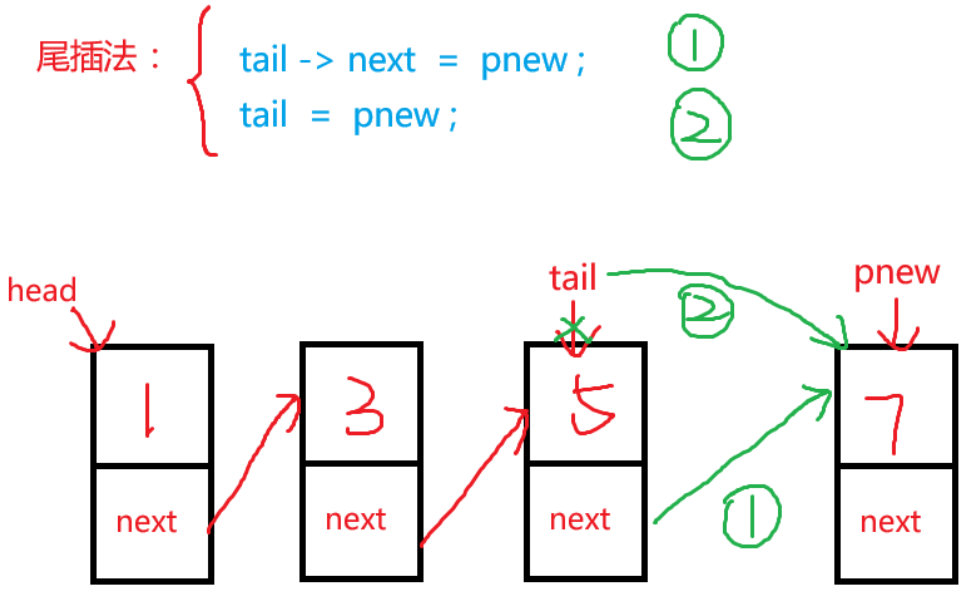
```
#include <stdio.h>

typedef struct node //数据结点
{
    int data;//数据域
    struct node *next;//指针域
}node;

int main()
{
    //创建4个结点
    node a,b,c,d;
    //为每个结点赋值以及关系的建立
    (&a)->data = 1;
    (&a)->next = &b;
    (&b)->data = 3;
    (&b)->next = &c;
    (&c)->data = 5;
    (&c)->next = &d;
    (&d)->data = 7;
    (&d)->next = NULL;
    //定义一个head指针保存第一个元素地址
    node *head = &a;   //head = &a

    node*temp = head;
    while(temp!=NULL)
    {
```

```
        //重复的代码
        printf("%d ",temp->data);
        temp = temp->next;//访问下一个元素
    }


}
```

保存数据，一般使用动态(malloc)的方式创建每一个结点

```
动态创建单链表结点

//从键盘输入
int num;
scanf("%d",&num);

//动态开辟空间
node *pnew= (node *)malloc(node);
pnew->data = num;
pnew->next = NULL;
```

尾插法




```
node *head = NULL;//头指针
node *tail = NULL;//尾指针

//创建新的结点

if(head == NULL)//没有数据
{
    head = tail = pnew;//新的节点成为第一个节点和最后一个节点
}
else//有数据结点
{
    tail->next = pnew;//尾巴的下一个结点是新的结点
```

```
    tail = pnew;//新的结点成为尾巴
}
```

```
假设数据元素类型elemtype(数据的类型)
typedef int elemtype;//给int创建一个别名叫做elemtype
typedef struct node
{
    elemtype data;
    struct node*next;
}node;
```

1)根据用户输入的数据，创建一个单链表(节点的顺序，就是用户输入的顺序)

listlist.c/linklist.h/main.c

```
//linklist.h
#ifndef _LINKLIST_H_
#define _LINKLIST_H_

typedef int elemtype;//给int创建一个别名叫做elemtype
typedef struct node
{
    elemtype data;
    struct node*next;
}node;

node *create_list();
void print_list(node *head);

#endif
```

```
//linkst.c
#include "linklist.h"
#include <stdio.h>
//创建链表
node *create_list()
{
    node *head = NULL;//头指针
    node *tail = NULL;//尾指针

    while(1)
    {
        elemtype d;
        //1.输入数据     (约定输入0结束循环)
        scanf("%d",&d);
        if(d == 0)
            break;
        //2.为你的数据创建一个节点
        node *pnew = (node*)malloc(sizeof(node));
        pnew->data = d;
        pnew->next = NULL;
        //3.把新的结点插入到链表中(尾插法)
        if(head == NULL)//没有数据结点
        {
            head=tail=pnew;//新的结点会成为头和尾
        }
```

```
        else//有数据结点
        {
            tail->next = pnew;//tail的下一个为pnew
            tail = pnew;//pnew成为尾巴
        }
    }

    return head;
}

//打印链表
void print_list(node *head)
{
    node *temp = head;//移动指针

    while(temp!=NULL)
    {
        printf("%d ",temp->data);//打印当前这个元素的数据
        temp = temp->next;//访问temp的下一个元素
    }
    printf("\n");
}
```

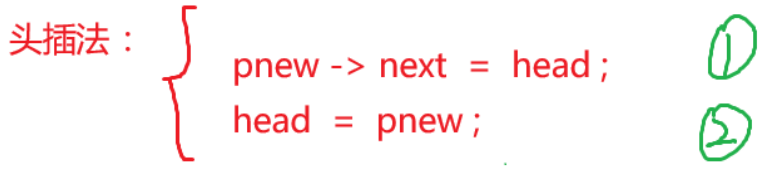
```
//main.c
#include <stdio.h>
#include "linklist.h"
int main()
{
    //创建链表
    node *head = create_list();

    //打印链表
    print_list(head);
}
```

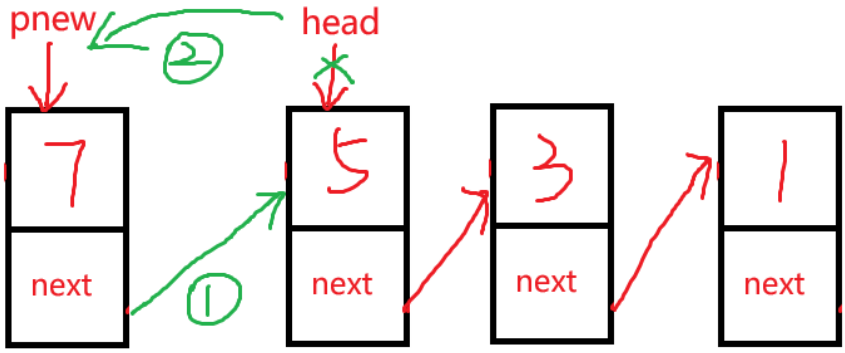
2.写一个尾插法的函数，在创建链表函数中进行调用

```
//尾巴的性质：尾巴的下一个next
/*
    尾插法
    head:保存链表的第一个节点的地址
    d：需要插入的数据的值
    返回值：返回链表的第一个节点的地址
*/
node *tail_insert(node *head,elemtype d)
{
    //1.为d创建一个新的结点

    //2.插入
    if(head == NULL)//没有数据结点
    {
        head=pnew;//新的结点会成为头
    }
    else//有数据结点
    {
```

```
        //2.1 找尾巴tail

        //2.2 把pnew插入到最后
        tail->next = pnew;//最后一个结点的下一个为pnew
    }

    //返回第一个结点的地址
    return head;
}
```

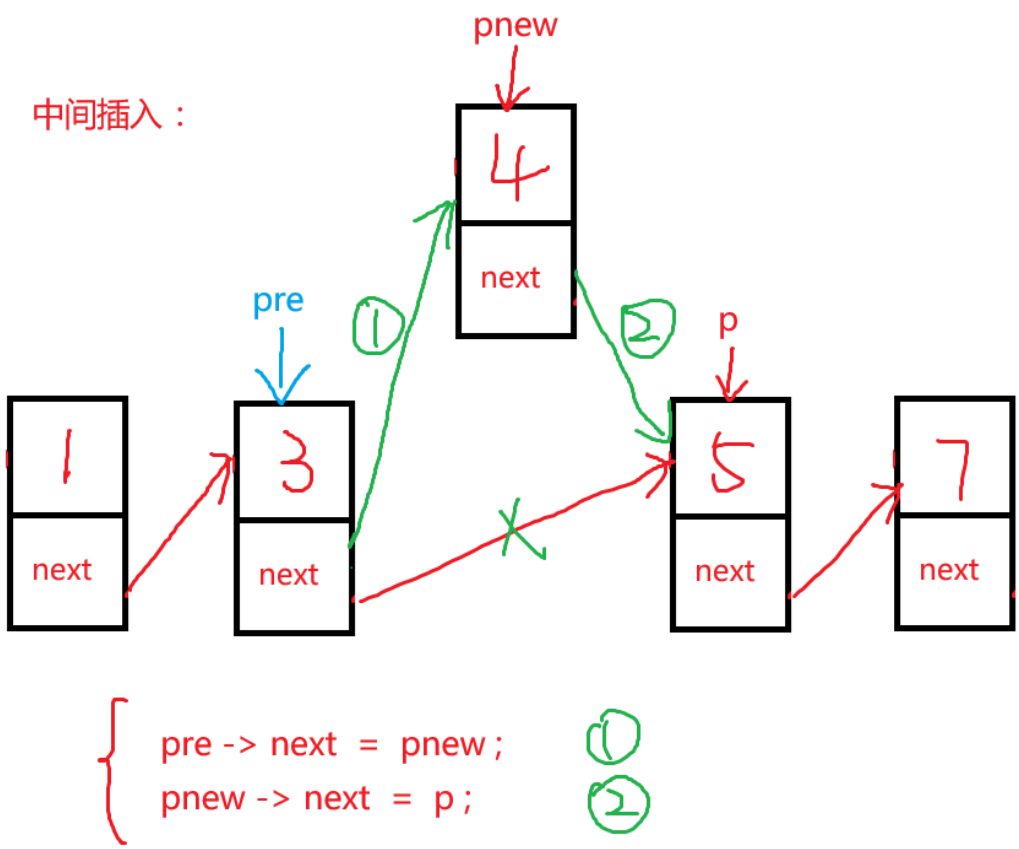
3.写一个头插法的函数，在创建链表函数中进行调用






```
/*
    头插法
    head:保存链表的第一个节点的地址
    d：需要插入的数据的值
    返回值：返回链表的第一个节点的地址
*/
node *head_insert(node *head,elemtype d)
{
    //1.为d创建一个新的结点

    //2.插入(头插法)
    if(head == NULL)
    {
        head=pnew;//新的结点会成为头
    }
    else
    {
        //新结点的下一个是head
        //新结点成为第一个结点
    }
}
```

4.在链表中查找一个值为x的节点，在值为x的结点前面添加一个值为a的结点，如果没有找到x，把a加入到链表的最后，并将新链表返回

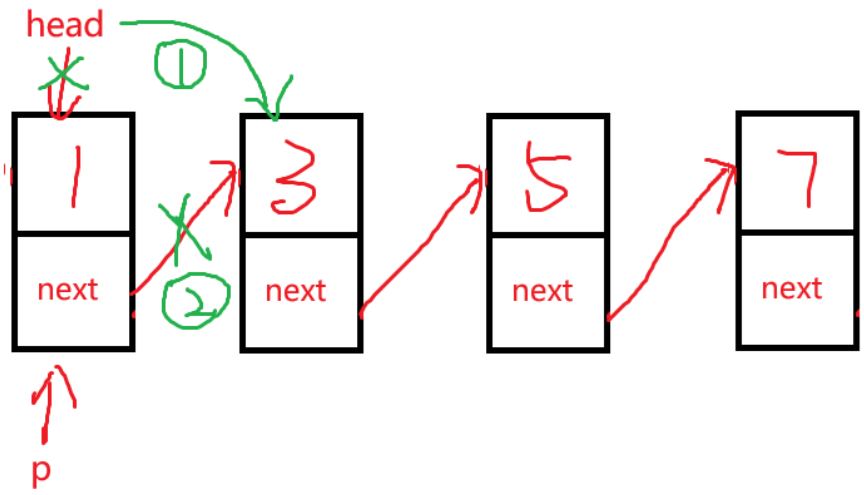



```
node *add_a_node(node *head,elemtype x,elemtype a)
{
    //1.为a创建一个新的结点

    //2.找值为x的结点(遍历)
    node *p = head;//p找值为x的结点
    node *pre = NULL;//保存p的前一个结点
    while(p!=NULL)
    {
        if(p->data == x)//找到了
            break;
        //没找到
        pre = p;//p会成为前一个结点
        p=p->next;//找下一个结点
    }

    //3.找到了if(p!=NULL)
        //3.1 p==head   头插
        //3.2 p!=NULL   中间插

    //4.没找到   尾插

    return head;
}
```

5.在链表中找到值为x的结点，将其删除，如果有多个值为x的结点，就删除第一个，如果没有找到就不删除

删除首节点：

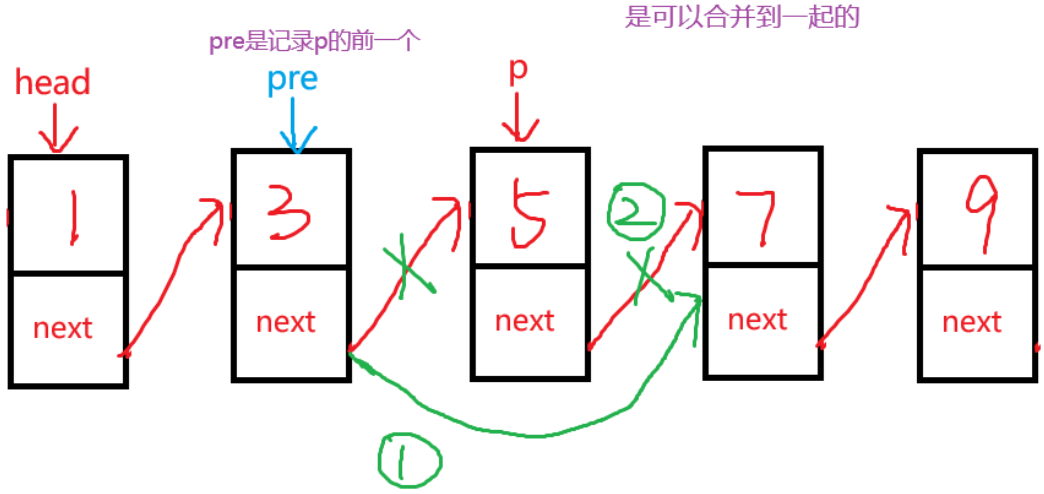
```
head = head -> next ;
p -> next = NULL ;
free( p ) ;
```



删除 中间节点 和 末尾节点：

```
pre -> next = p -> next ;
p -> next = NULL ;
free( p ) ;
```

删除 中间节点 和 删除末尾节点 的操作，是可以合并到一起的




```
node *delete_node(node*head,elemtype x)
{
    //1.找值为x的结点

    //2.删除值为x的结点(分情况讨论 头删 中间删和尾删统一为中间删的代码)
    //找到了 if(p!=NULL)
    //情况1：找到的节点p就是head  头删
    //情况2：中间删
}
```

6.写一个函数，销毁链表

```
//循环头删 把每一个结点都要释放麻醉后打印链表的结果为NULL
node *destroy_list(node *head)
{
    node *p = head;//p保存要删除的结点的地址

    while(p)
    {
        head = head->next;
        //孤立p
        p->next = NULL;
        free(p);
        p = NULL;

        p = head;//p保存下一次要删除的结点
    }

    return head;
}
```

7.根据用户的输入创建一个升序链表，约定输入0表示结束

```
node *create_list_asc()
{
    node *head = NULL;

    while(1)
    {
        //1.输入数据(约定输入0结束循环)

        //2.创建结点pnew

        //3.把新的结点插入到到链表中
        //3.1 找位置 找第一个比pnew大的结点p
        //3.2 插入
        //如果找到了 头插(p==head)  中间插
        //如果没找到  尾插

    }

    return head;
}
```

8.写一个函数，求单链表结点的数目

```
int get_node_count(node *head)
{

}
```

9.就地逆置一个单链表(不能额外申请空间)，将新链表返回

```
1 3 5 7 9 原来的链表
9 7 5 3 1 逆置后的链表
```

```
思路:
    把原来链表中的每一个结点 摘下来(相当于头删 但是不用free)
    然后按照 头插法 插入到新链表

node *h = NULL;//指向新的链表
node *p = head;//p保存要摘下来的那个节点

1.摘下来
    p=head;
    head = head->next;
    p->next = NULL
2.把节点p使用头插 放到h链表中
    if(h == NULL)
        h = p;
    else
    {
        p->next = h;
        h = p;
    }
3.什么时候结束循环? head == NULL 结束


node *reverse_list(node *head)
{
    node *h = NULL;//指向新的链表
    node *p = head;//p保存要摘下来的那个节点

    while(head != NULL)
    {
        //1.摘下来
        p=head;
        head = head->next;
        p->next = NULL;
        //2.把节点p使用头插 放到h链表中
        if(h == NULL)
            h = p;
        else
        {
            p->next = h;
            h = p;
        }
    }

    return h;
}
```